

Appunti Esame

Status Not started

Intro all'ingegneria del software

Cos'è l'ingegneria del software

Definizione: è l'applicazione di un approccio sistematico, disciplinato e quantificabile allo sviluppo, all'esercizio e alla manutenzione del software.

- **Obiettivo:** creare prodotti software economici, efficienti e di alta qualità, applicando conoscenze scientifiche a problemi pratici.
- **Differenza con l'Informatica:** mentre l'informatica si concentra su teoria e fondamentali, l'ingegneria del software si orienta ai problemi pratici di sviluppo e consegna di prodotti utili.

Il ciclo di vita e le attività

Le attività fondamentali sono:

- **Specifica:** definizione dei requisiti e dei vincoli del sistema.
- **Progettazione (design):** definizione dell'architettura e dei componenti.
- **Implementazione:** stesura del codice.
- **Validazione:** verifica che il sistema soddisfi le richieste (testing).
- **Manutenzione:** modifica del software dopo la consegna per correggere errori o adattarlo a nuove esigenze.
- **Nota sui costi:** la maggior parte del costo software non è nello sviluppo, ma nell'evoluzione/manutenzione (circa il 75% o più).

Qualità del software

- **Definizione di qualità:** la totalità delle caratteristiche di un prodotto basate sulla sua abilità di soddisfare bisogni espliciti ed impliciti.
- **Correttezza (correctness):** un software è corretto se soddisfa le specifiche funzionali.
 - *Attenzione:* se le specifiche sono errate, il software può essere "corretto" rispetto alle specifiche ma inutile per l'utente.
- **Dependability (affidabilità/sicurezza):** include concetti come *reliability* (affidabilità nel tempo) e *safety/security* (sicurezza da danni o accessi).
- **Manutenibilità:** la facilità con cui il software può essere modificato per nuove esigenze.
- **Usabilità:** la facilità di utilizzo per gli utenti previsti.
- **Efficienza:** evitare sprechi di risorse (memoria, processore).

Caratteristiche intrinseche del software

- **Invisibilità:** il software è immateriale e non può essere rappresentato completamente da un unico schema geometrico.
- **Complessità:** numero di stati enorme, assenza di leggi naturali che lo regolano.
- **Conformità:** il software deve adattarsi (conformarsi) all'hardware e alle procedure umane, non viceversa.
- **Modificabilità (software):** è soggetto a continui cambiamenti perché le esigenze mutano costantemente.

"Miti" del software

- **Mito del management:** "se siamo in ritardo, aggiungere persone al progetto ci farà recuperare". (⇒ spesso rallente ulteriormente).
- **Mito del cliente:** "una descrizione generica basta per iniziare a programmare, i dettagli si vedono dopo". (⇒ causa costi enormi di rifacimento).
- **Mito del programmatore:** "il lavoro finisce quando il codice gira" o "la qualità si vede solo a programma finito". (⇒ la qualità va costruita durante tutto il processo).

Glossario rapido

- **Stakeholder:** le parti in causa (cliente, utente, sviluppatore).
- **Sistemi legacy:** sistemi vecchi ma indispensabili che devono essere mantenuti.
- **Specifica vs Implementazione:** la specifica dice cosa fare (esterno), l'implementazione dice come farlo (interno/codice).

Gestione progetti e processi di sviluppo

Gestione del progetto e scheduling

- **Vincoli del manager:** un manager deve gestire il progetto con vincoli di budget (personale costoso non sempre disponibile) e disponibilità di personale qualificato.
- **Milestones vs Deliverables:**
 - **Milestone:** è il punto finale di un'attività del processo, un traguardo interno verificabile.
 - **Deliverable:** è un risultato tangibile del progetto che viene consegnato al cliente.
- **Gantt vs PERT:**
 - **Diagramma di Gantt:** si concentra sul calendario (asse temporale). Mostra la durata delle attività e la loro sequenza nel tempo, utile per pianificare e tracciare. Non visualizza bene le interdipendenze.
 - **Diagramma PERT (rete delle attività):** evidenzia le **interdipendenze** tra i task (quale task deve finire prima che inizi l'altro) e permette di calcolare il percorso critico (il tempo minimo necessario per finire il progetto).

Ciclo di vita del software

Le 5 attività fondamentali sono:

1. **Analisi dei requisiti:** cosa deve fare il software.
2. **Progettazione (Design):** come lo deve fare (architettura, componenti).
3. **Implementazione (codifica):** scrittura del codice.
4. **Convalida (testing):** verifica che funzioni.
5. **Manutenzione:** evoluzione dopo la consegna.

Modelli di sviluppo

Modello a cascata (waterfall)

È il modello classico:

- **Funzionamento:** sequenziale. Si passa alla fase successiva solo quando la precedente è finita e approvata ("firmata").
- **Vantaggi:** alta qualità del codice, tanta documentazione, chiaro e disciplinato. Utile per progetti grandi con requisiti stabili.
- **Svantaggi:** poche interazioni con il cliente (solo all'inizio), difficoltà a gestire i cambiamenti in corso d'opera, tempi lunghi per vedere il prodotto finito.
- **Variante V-Model:** mette in relazione ogni fase di sviluppo (sinistra della V) con la corrispondente fase di test (destra della V).

Modello a spirale (Boehm)

- **Caratteristica chiave:** è guidato dall'**analisi dei rischi**. Ogni giro della spirale valuta i rischi prima di procedere.

Unified process (UP)

- Processo iterativo diviso in 4 fasi temporali: **inizio (inception)**, **elaborazione**, **costruzione**, **transizione**.

Metodologie Agili (XP e Scrum)

- **Principi Agile:** il software funzionante è più importante della documentazione; la collaborazione col cliente è più importante del contratto; rispondere al cambiamento è più importante di seguire un piano.
- **Extreme Programming (XP):**
 - Focalizzato su **piccole release** (ogni 2 settimane) e feedback rapido.
 - **Pair programming:** due programmatori lavorano insieme su un computer (uno scrive, l'altro controlla/pensa).

- **Refactoring:** migliorare la struttura del codice senza cambiarne il comportamento esterno.
- **Test-Driven Development (TDD):** i test vengono scritti prima del codice.
- **Scrum:**
 - Basato su iterazioni chiamate **Sprint** (di solito 2-4 settimane).
 - **Ruoli:** product owner (voce del cliente), Scrum Master (facilitatore), Team di sviluppo.
 - **Artefatti:** product backlog (lista di tutto il lavoro), Sprint Backlog (lavoro dello sprint corrente).

Altri concetti

- **User Story:** un modo "agile" di scrivere i requisiti. Sono brevi descrizioni di funzionalità dal punto di vista dell'utente (es. su una card di carta).
- **Verifica vs Convalida:**
 - *Verifica:* stiamo costruendo il prodotto nel modo giusto? (rispetto alle specifiche).
 - *Convalida:* stiamo costruendo il prodotto giusto? (rispetto ai bisogni dell'utente).
- **Testing:**
 - *Unit test:* singoli componenti.
 - *Integration test:* come i componenti lavorano insieme.
 - *System test:* intero sistema.
 - *Acceptance test (alpha/beta):* test con i dati del cliente.

Git

Che cos'è Git?

Git è un software di **controllo di versione (VCS)** creato da Linus Torvalds.

- **Scopo:** serve a tenere traccia di tutte le modifiche al codice sorgente durante lo sviluppo.
- **Architettura decentralizzata:** a differenza dei sistemi centralizzati, Git è distribuito. Non richiede un server centrale obbligatorio; ogni sviluppatore ha una copia completa della base di dati e dei file sulla propria macchina locale.
- **Vantaggio:** permette di lavorare in parallelo senza essere costantemente connessi a internet.

Architettura e aree di lavoro

Per capire come muovere i file, bisogna visualizzare tre aree principali in cui i file possono trovarsi:

1. **Working Directory:** la cartella locale dove stai lavorando e modificando i file attualmente.
2. **Staging Area (o Index):** una zona di "preparazione" dove metti i file pronti per essere salvati.
3. **Repository (local repo):** il database dove le modifiche vengono salvate definitivamente (congelate).

Esiste poi il **remote repository** (es. GitHub), che è il server remoto per la condivisione del codice.

Ciclo di vita dei file

Git agisce come un guardiano che osserva i file nella cartella del progetto. I file possono assumere stati specifici. Gli stati principali sono:

- **Untracked (non tracciato):** il file esiste nella cartella ma Git non lo sta monitorando.
- **Modified (modificato):** il file è stato cambiato rispetto all'ultimo salvataggio, ma non è ancora pronto per il commit.
- **Staged:** il file è stato marcato (con `git add`) per essere incluso nel prossimo salvataggio. **Nota:** solo i file in questa area finiranno nel commit.
- **Committed:** il file è stato congelato e salvato nel database locale di Git.

Branching

Lo sviluppo in Git non è lineare ma si basa sui rami (**branch**).

- **Master:** è solitamente il ramo principale di partenza.

- **Commit:** è la registrazione di un nuovo stato, un'istantanea del lavoro svolto in un preciso momento.
- **Parallelismo:** gli sviluppatori possono creare branch paralleli (es. per correggere un bug o aggiungere una feature) senza intaccare il ramo `master`.

Workflow avanzato: in progetti complessi si usano rami specifici come:

- Feature branches: per nuove funzionalità.
- Develop: per lo sviluppo continuo.
- Release: per preparare una nuova versione.
- Hotfixes: per correzioni urgenti di bug in produzione.
- Tags: etichette per segnare versioni specifiche.

Merging

Quando un lavoro su un branch secondario è finito, deve essere unito al ramo principale (`master`) tramite il **merge**.

Procedura di merge:

1. Spostarsi sul ramo di destinazione (`git checkout master`).
2. Eseguire il comando di unione (es. `git merge idea`).
3. **Gestione dei conflitti:** se le stesse righe sono state modificate diversamente, Git non può decidere da solo. Spetta allo sviluppatore risolvere il conflitto manualmente decidendo cosa tenere e cosa scartare.

Comandi principali

- `git init`: inizializza un nuovo repository.
- `git add`: sposta i file dalla working directory alla staging area (da *modified* o *untracked* a *staged*).
- `git commit`: salva i file dalla staging area al local repo (crea uno snapshot).
- `git push`: invia i commit dal local repo al remote repo.
- `git pull`: scarica le modifiche dal remote repo e le unisce al tuo lavoro locale.
- `git checkout`: serve per spostarsi tra i rami.
- `git log --graph --all --oneline`: visualizza la storia dei commit e l'evoluzione dei rami in modo grafico e testuale.

Architetture per il web

Protocolli e stili architetturali

RPC (Remote Procedure Call)

- **Concetto:** è il modo più semplice di pensare ai servizi web. Si chiama una funzione su un server remoto come se fosse locale.
- **Funzionamento:** di solito c'è un singolo URL (endpoint). La richiesta specifica il *nome della funzione* e i *parametri*.
- **Esempio:** una chiamata a `flickr.photos.search` passando i tag.

SOAP (Simple Object Access Protocol)

- **Definizione:** un protocollo basato su XML per lo scambio di informazioni strutturate.
- **Caratteristiche:**
 - Molto rigoroso e formale.
 - Usa WSDL (Web Service Description Language) per descrivere il servizio (quali metodi esistono, quali dati accettano).
 - **Vantaggio:** ottimo supporto per generare automaticamente il codice client/server (tramite il WSDL).
 - **Svantaggio:** "pesante" (verboso) a causa dell'XML.

REST (REpresentational State Transfer)

- **Definizione:** non è un protocollo, ma uno **stile architetturale**.
- **Concetti chiave:**

- **Risorse:** ogni dato è una “risorsa” identificata da un **URI** unico (es. `/users/123`).
- **Verbi HTTP:** usa i metodi standard del protocollo HTTP per le operazioni CRUD:
 - **GET:** leggere dati (senza modificarli).
 - **POST:** creare una nuova risorsa.
 - **PUT:** aggiornare una risorsa esistente (invia l'intera risorsa).
 - **DELETE:** eliminare una risorsa.
 - *(PATCH: aggiornamento parziale).*
- **Rappresentazione:** i dati possono viaggiare come JSON (più comune oggi) o XML.
- **Stateless:** il server non deve memorizzare lo stato del client tra due richieste.

Pattern architetturali (MVC e MVP)

Questi sono pattern fondamentali per separare le responsabilità nel codice (separation of concerns).

MVC (Model-View-Controller)

Il pattern più comune per le interfacce utente e il web.

1. **Model (modello):** gestisce i dati e la logica di business (es. accesso al database).
 2. **View (vista):** si occupa della **visualizzazione** (HTML/CSS). Mostra i dati all'utente.
 3. **Controller:** l'intermediario.
 - Riceve l'input dall'utente (es. click, richiesta HTTP).
 - Manipola il model.
 - Aggiorna la view.
- **Flusso web (PHP example):**
 - *Router (front controller):* riceve l'URL (es. `/users/show/1`), capisce quale Controller chiamare.
 - *Controller:* chiama il model per avere i dati dell'utente ID 1.
 - *View:* riceve i dati dal controller e genera la pagina HTML.

MVP (Model-View-Presenter)

Una variazione dell'MVC, usata quando la view è “passiva”.

- **Presenter:** sostituisce il controller. Tutta la logica di presentazione sta qui.
- **Differenza principale:** in MVC, la View può talvolta osservare direttamente il Model. In MVP, il Presenter fa da “ponte” totale: prende i dati dal Model, li formatta e li spinge dentro la View. La view non tocca mai il model.

Quando usare (e non usare) MVC

- **Quando usarlo:**
 - Applicazioni complesse o grandi (e-commerce).
 - Necessità di testare separatamente logica e interfaccia.
 - Team numerosi (chi fa il backend non pesta i piedi a chi fa il frontend).
 - Se l'interfaccia cambia spesso ma i dati no.
- **Quando non usarlo:**
 - Applicazioni molto semplici o piccole (aggiunge complessità inutile).
 - Applicazioni real-time (giochi, chat) dove la latenza dell'architettura MVC potrebbe essere un problema.
 - Se UI e Logica sono troppo accoppiate.

Requisiti e UML

Fondamenti dell'ingegneria dei requisiti

- **Analisi vs Specifica**
 - **Analisi:** processo di studio del dominio applicativo. Serve a capire cosa serve e il contesto (ambiente).
 - **Specifica (SRS):** il risultato dell'analisi. È un documento (o modello) che deve essere preciso, completo e verificabile.
- **Classificazione dei requisiti:**
 - **Funzionali:** cosa il sistema *fa* (input → output, servizi). Se soddisfatti, il sistema è "corretto".
 - **Non-Funzionali (Quality Requirements):** come il sistema *si comporta*. Se non soddisfatti, il sistema è spesso inutile.
 - *Di Prodotto:* affidabilità, usabilità.
 - *Organizzativi:* standard da seguire, tempi di consegna.
 - *Esterni:* leggi, privacy, interoperabilità.
 - **Vincoli (constraints):** limitazioni imposte al design (es. hardware specifico, budget).
- **Terminologia critica (safety vs security):**
 - **Safety (innocuità):** il sistema non deve causare danni a persone/cose.
 - **Security (riservatezza):** protezione da accessi non autorizzati.
- **Sistemi real-time:** prevedibile (rispetto rigoroso dei vincoli temporali).

Convalida e formalismo

Per assicurarci che i requisiti siano giusti:

- **Criteri di qualità:** completezza (non manca nulla), consistenza (non ci sono contraddizioni), tracciabilità (requisito ↔ codice ↔ test).
- **Tecniche di convalida:** revisioni (lettura critica), prototipazione (feedback utente), generazione di test (se un requisito non è testabile, non è valido).
- **Livelli di formalità:**
 - *Informali:* linguaggio naturale (ambiguo).
 - *Semiformali (UML):* sintassi grafica rigorosa, semantica a volte aperta.
 - *Formali:* matematica pura (es. automi a stati finiti), non ambigui e verificabili.

Modellazione strutturale

Vista statica: Diagramma delle classi (com'è fatto)

Elementi base:

- **Classe (rettangolo):** rappresenta un concetto (es. `Studente`).
 - *Parte superiore:* nome.
 - *Parte centrale (attributi):* i dati (es. `- matricola: int`).
 - *Parte inferiore (operazioni):* le funzioni (es. `+ iscriviti()`).
 - **Nota sulla visibilità:** `+` (pubblico, tutti vedono), `-` (privato, solo la classe vede), `#` (protetto, visibile ai figli).

Le relazioni:

1. **Associazione (linea semplice):** "c'è un collegamento". Ed. un docente usa un computer.
2. **Aggregazione (◇):** relazione "Parte-Tutto" debole.
 - L'oggetto parte può vivere anche senza l'oggetto tutto.
 - **Squadra ◇ — Giocatore.** Se sciogli la squadra, il giocatore esiste ancora (è libero).
3. **Composizione (◆):** relazione "Parte-Tutto" forte.
 - Se muore il contenitore, muore il contenuto.
 - **Casa ◆ — Stanza.** Se demolisci la casa, la stanza non esiste più da sola.
4. **Generalizzazione (freccia triangolare vuota △):** relazione "è un..." (ereditarietà).

- `Studente` è una `Persona`.
- *Regola d'oro (Liskov)*: ovunque nel codice usi `Persona`, devi poter usare `Studente` senza che il programma esploda.

Vista funzionale: Casi d'uso (cosa fa per l'utente)

Questo diagramma serve a definire il perimetro (cosa fa il software e cosa no).

- **Attore (omino)**: chi usa il sistema. Attenzione: può essere una persona (`Cliente`) o un altro software (`Sistema di Pagamento esterno`).
- **Use case (ovale)**: una funzione che ha valore per l'attore (es. `Prelevare Contanti`).
- **Relazioni difficili**:
 - `<<include>>` (**obbligatorio**): "Per fare A devo per forza fare B".
 - Es. `Bonifico` `<<include>>` `Verifica Saldo`. Non puoi fare bonifico senza verifica.
 - `<<extend>>` (**opzionale**): "faccio A, e se capita faccio anche B".
 - Es. `Login` `<<extend>>` `Mostra Errore`. L'errore lo mostri solo se la password è sbagliata.

Vista dinamica: come si comporta nel tempo

Ci sono due diagrammi principali:

Diagramma di sequenza (chi parla con chi?)

Mostra lo scambio di messaggi nel tempo cronologico (dall'alto in basso).

- **Lifeline (linea tratteggiata)**: la vita dell'oggetto.
- **Messaggi**:
 - *Freccia piena*: **sincrono** (telefono a Mario e aspetto in linea finché non risponde).
 - *Freccia aperta*: **asincrono** (mando una mail a Mario e vado a fare altro).
- **Frammenti (blocchi logici)**:
 - `alt`: se succede questo fai X, altrimenti fai Y (If-Else).
 - `opt`: fai X solo se serve (If senza Else).
 - `loop`: ripeti X finché... (ciclo while/for).

Diagramma di stato (Statecharts)

Fondamentale per i sistemi che "reagiscono" (es. ascensore, bancomat). Descrive come un oggetto cambia stato.

- **Sintassi della transizione**: `Evento [Guardia] / Azione`
 - *Evento*: cosa succede? (Es. `PulsantePremuto`)
 - *Guardia*: condizione tra parentesi quadre (es. `[PortaChiusa]`). Se la porta è aperta, l'evento viene ignorato.
 - *Azione*: cosa fa il sistema? (Es. `AvviaMotore`)
- **Azioni interne allo stato**:
 - `entry`: cosa faccio appena entro nello stato.
 - `do`: cosa faccio *continuamente* mentre sono lì.
 - `exit`: cosa faccio prima di andarmene.

Vista fisica: dove gira (Deployment & Componenti)

- **Componenti**: sono i blocchi di codice (moduli, librerie). Si collegano tramite interfacce "a lecca-lecca" (cerchio = offro un servizio, semicerchio = mi serve un servizio).
- **Deployment**: mappa il software sull'hardware.
 - *Nodo*: il computer fisico (server, smartphone).
 - *Artefatto*: il file reale (`app.exe`, `database.sql`) che metti dentro il nodo.

Design patterns

Concetti fondamentali: Architettura vs Design Patterns

È cruciale distinguere tra il livello macro (architettura) e il livello micro (pattern).

- **Architettura software:** è la struttura di alto livello del sistema. Definisce i componenti principali (sottosistemi), le loro relazioni e le proprietà globali (scalabilità, sicurezza).
- **Design pattern:** è una soluzione riutilizzabile a un problema ricorrente in un contesto specifico. Si applica a livello di classi e oggetti (grana fine).
 - *Elementi di un pattern:* nome, problema, soluzione (spesso un diagramma UML), conseguenze (pro/contro).

Stili architetturali (architectural patterns)

Non si inventa l'architettura da zero; si usano stili consolidati.

- **Layered (a strati):** organizzazione gerarchica. Ogni strato fornisce servizi a quello superiore e usa quello inferiore.
 - *Esempio:* presentation → business logic → data access.
 - *Vantaggi:* basso accoppiamento, facile sostituire uno strato.
- **Client-Server:** distribuzione del carico. Il server offre servizi, il client li richiede.
- **MVC (Model-View-Controller):** fondamentale per interfacce utente.
 - **Model:** dati e logica di business.
 - **View:** visualizzazione all'utente.
 - **Controller:** gestisce l'input utente e aggiorna Model e View.
 - *Obiettivo:* separare la logica dalla presentazione.

Design patterns (GoF - Gang of Four)

I pattern sono divisi in tre categorie.

Pattern creazionali (creazione di oggetti)

Gestiscono il processo di istanziazione, rendendo il sistema indipendente da *come* gli oggetti vengono creati.

1. Singleton:

- *Scopo:* garantiscono che una classe abbia una sola istanza e fornire un punto di accesso globale ad essa.
- *Uso:* database connection, Logger, Configuration manager.
- *UML:* costruttore privato, attributo statico `instance`, metodo statico `getInstance()`.

2. Factory method:

- *Scopo:* definire un'interfaccia per creare un oggetto, ma lasciare alle sottoclassi la decisione su quale classe istanziare.
- *Principio:* delega la creazione. Il client lavora con l'interfaccia (es. `Prodotto`), non con la classe concreta.

3. Abstract factory:

- *Scopo:* creare famiglie di oggetti correlati senza specificare le loro classi concrete (es. creare widget per Windows o Mac: `BottoneWin`, `FinestraWin` vs `BottoneMac`, `FinestraMac`).

Pattern strutturali (composizione di classi/oggetti)

Si occupano di come comporre classi e oggetti per ottenere strutture più grandi.

1. Adapter:

- *Scopo:* convertire l'interfaccia di una classe in un'altra attesa dal client. Fa collaborare classi incompatibili.
- *Analisi:* è come l'adattatore della presa elettrica.
- *UML:* il `Client` chiama `Target.request()`, l'`Adapter` traduce chiamando `Adaptee.specificRequest()`.

2. Composite:

- *Scopo:* trattare oggetti singoli e composizioni di oggetti nello stesso modo (struttura ad albero).
- *Esempio:* file system (file e cartelle sono trattati ugualmente come "nodi").

Pattern comportamentali (interazione e responsabilità)

Si occupano della comunicazione tra oggetti e dell'assegnazione delle responsabilità.

1. Observer (fondamentale):

- *Scopo*: relazione uno-a-molti. Quando un oggetto (subject) cambia stato, tutti i dipendenti (observers) vengono notificati e aggiornati automaticamente.
- *Esempio*: Excel (cambia i dati, il grafico si aggiorna).
- *Meccanismo*: `attach()`, `detach()`, `notify()`.

2. Command:

- *Scopo*: incapsulare una richiesta come un oggetto. Permette di parametrizzare clienti con richieste diverse, accodare richieste o supportare operazioni di **undo**.
- *Componenti*: `Invoker` (chi chiama), `Command` (interfaccia), `ConcreteCommand` (implementazione), `Receiver` (chi fa il lavoro).

3. Strategy:

- *Scopo*: definire una famiglia di algoritmi, incapsularli e renderli intercambiabili. Permette all'algoritmo di variare indipendentemente dal client che lo usa.
- *Esempio*: un navigatore che sceglie il percorso "più veloce", "più corto" o "senza pedaggi" (cambia solo la strategia, non l'oggetto navigatore).

4. State:

- *Scopo*: permettere a un oggetto di cambiare comportamento quando il suo stato interno cambia. Sembrerà che l'oggetto abbia cambiato classe.
- *Uso*: implementazione pulita di macchine a stati finiti (evitando infiniti `if-else` o `switch`).

Principi di progettazione sottostanti

Perchè si usa un pattern? La risposta è in questi principi:

- **Favorire la composizione rispetto all'ereditarietà**: l'ereditarietà è statica e rigida; la composizione (oggetto che contiene un altro oggetto) è dinamica e flessibile.
- **Programmare verso un'interfaccia, non un'implementazione**: rende il codice disaccoppiato. Il client non deve sapere quale classe specifica sta usando, basta che rispetti il contratto (interfaccia).
- **Open/Closed principle**: il software deve essere aperto all'estensione (nuovi pattern/funzioni) ma chiuso alla modifica (non rompere il codice esistente).

Testing

Concetti fondamentali: Verifica e Validazione

- **Verifica (verification)**: "stiamo costruendo il prodotto nel modo giusto?"
 - **Obiettivo**: conformità alle specifiche.
- **Validazione (validation)**: "stiamo costruendo il prodotto giusto?"
 - **Obiettivo**: soddisfare i bisogni reali dell'utente (anche se le specifiche erano tecnicamente errate ma eseguite alla lettera).

Tipologie di difetti: i difetti non nascono solo nel codice.

1. *Specifiche*: ambigue, omissioni (i più costosi da correggere se trovati alla fine).
2. *Progettazione*: algoritmi errati, interfacce sbagliate.
3. *Codifica*: errori di sintassi o logica.
4. *Testing*: test case sbagliati.

VERIFICA vs VALIDAZIONE

Verifica → "nel modo giusto"

Validazione → "il prodotto giusto"

Tecniche statiche vs dinamiche

È fondamentale distinguere tra analisi senza esecuzione e con esecuzione.

Tecniche statiche (senza esecuzione)

- **Ispezioni (software inspection):** revisione formale del codice o dei documenti da parte di un team.
 - *Metodo fagan:* ruoli precisi (autore, lettore, tester, moderatore). È molto efficace (trova il 60-90% dei difetti).
- Analisi statica automatizzata: tool che scansionano il codice sorgente alla ricerca di pattern di errore (variabili non inizializzate, loop infiniti) senza eseguirlo.

Tecniche dinamiche (testing)

Richiedono l'esecuzione del programma con dati di input scelti.

- **Definizione cruciale:**
 - **Difetto (Fault/Bug):** l'errore nel codice.
 - **Guasto (failure):** il comportamento anomalo osservabile durante l'esecuzione causato dal difetto.
 - *Nota:* non tutti i difetti provocano guasti (es. codice morto o percorsi mai eseguiti).

BUG vs FAILURE

Bug → errore latente

Failure → errore visibile

Livelli di testing (granularità)

1. **Unit testing (test di unità):** testa il singolo componente/modulo isolato.
2. **Integration testing:** verifica come i moduli interagiscono.
3. **System testing:** verifica il sistema completo rispetto ai requisiti (funzionali e non).
4. **Acceptance testing (collaudo):**
 - *Alpha Test:* utenti interni/simulati.
 - *Beta Test:* utenti reali in ambiente reale.

Strategie di integrazione

- **Top-Down:** si parte dal main e si scende.
 - *Pro:* testi subito l'architettura.
 - *Contro:* richiede **stub** (moduli falsi che simulano quelli di basso livello non ancora scritti).
- **Bottom-Up:** si parte dai moduli base (es. utility).
 - *Pro:* facile creare i test case.
 - *Contro:* richiede **driver** (programmi che chiamano i moduli) e vedi l'architettura completa solo alla fine.
- **Sandwich:** misto dei due.

STUB vs DRIVER

Stub → simula il chiamato

Driver → simula il chiamante

Tipologie di test: White-box vs Black-box

Black-box (funzionale)

Si guarda solo input e output, ignorando il codice interno.

- **Equivalence Partitioning (partizionamento in classi di equivalenza):**
 - Non puoi testare tutti gli input infiniti. Raggruppi gli input in classi dove ci si aspetta che il sistema si comporti allo stesso modo.
 - *Esempio:* se accetto numeri `1-100`, testo un valore valido (50) e due invalidi (0, 101).

White-box (strutturale)

Si basa sulla struttura logica del codice (branch, loop).

- **Control flow testing:** si usa il grafo di flusso di controllo.
- **Complessità ciclomatica (McCabe):** una metrica fondamentale.
 - *Formula:* $V(G) = E - N + 2$ (dove E = archi, N = nodi) oppure *Numero di condizioni (predicati) + 1*.
 - Indica il numero di cammini linearmente indipendenti nel codice. Più è alto, più il codice è difficile da testare e mantenere.

■ **McCabe:** saper calcolare la complessità guardando un grafo o un pezzo di codice (conta gli `if`, `while` + 1).

Processo e metriche

- **TDD (Test Driven Development):** scrivere il test prima del codice.
 - *Ciclo:* Red (fail) → Green (pass) → Refactor.
- **Regression testing:** rieseguire i test vecchi dopo ogni modifica per assicurarsi di non aver rotto cose che funzionavano.
- **Code Coverage (copertura):** % di codice eseguito dai test.
 - *Attenzione:* 100% copertura $\neq$ assenza di bug (potresti aver dimenticato di testare un caso logico, anche se hai eseguito tutte le righe).